There is no way to access non-static from static, because `$this` is not available (which is as it should be; object context code has state that class context does not, and such an operation would be unsafe). It's fine for a non-static method to call another static member; however,

```
class QuoteBot
{
    private static function getInsight(): string
    {
        return "I've always found that writing comes from a great inner
        pain." . PHP_EOL;
    }

    public function insight(): void
    {
        print self::getInsight();
    }
}

$bot = new QuoteBot();
$bot->insight();
```

```
Output:
I've always found that writing comes from a great inner pain.
```

The example above shows an alternative to using `$this::`. Previously, in the `Counter` class, we accessed a static property with `$this::`, in the `QuoteBot` example above, we use `self::` to access the static member `getInsight()`. Both are perfectly valid, though conventionally it's a little more common to see `self::` used in object context code (because less thought is required: `self` is always available and works identically).

Class Constants

We've seen how to define global constants in Chapter 2, "Data, Types, and Variables." Classes are also able to define constant properties, which are declared with the `const` keyword. Let's add one to `QuoteBot`:

```
    public const FILM = "Barton Fink";
```